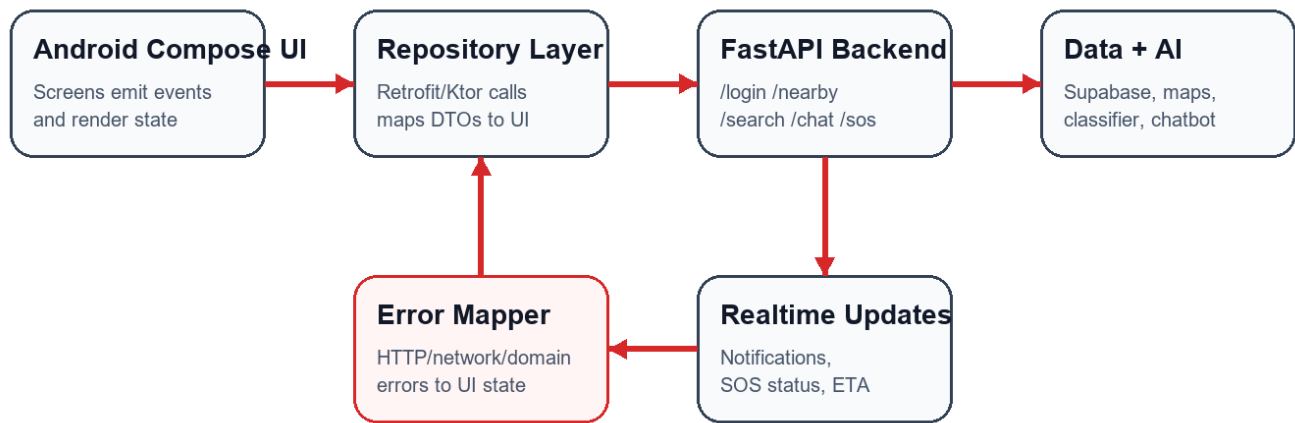


# RoadSOS Complete Project Documentation

Frontend progress, screen specifications, error handling, and backend integration handoff

Generated on 18 May 2026, 12:57 AM from the local repository. Android source folder was not modified.

## RoadSOS Backend Integration Flow



## Executive Summary

RoadSOS is a Kotlin + Jetpack Compose + Material 3 Android emergency assistance frontend. The current implementation is hackathon-ready and backend-ready: the UI flows, navigation, empty states, simulated errors, search/filter behavior, profile editing, permissions journey, and emergency/SOS surfaces are already implemented on the client side using local Compose state. Backend integration now needs to replace hardcoded data and simulated responses with authenticated API calls, persistent storage, live location, service discovery, AI chat, notifications, and SOS dispatch workflows.

Important current-state note: the frontend currently stores login, contacts, services, profile, notifications, and AI chat in memory. That makes the UI demonstrable, but data resets when the process restarts. The backend should expose stable contracts and the Android side should introduce a repository/ViewModel layer before production.

## Technology Stack

| Layer        | Current implementation                                                  | Backend handoff meaning                                                               |
|--------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Android UI   | Kotlin, Jetpack Compose, Material 3                                     | Backend can focus on JSON contracts while Android binds responses into Compose state. |
| Architecture | Composable screens, local remember/mutableStateOf, manual screen router | Add ViewModels, repositories, DTOs, and sealed UI states without redesigning screens. |

| Layer   | Current implementation                                                                             | Backend handoff meaning                                                                                                 |
|---------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Theme   | Custom dark theme, red emergency accent, reusable colors                                           | Backend severity/status can map to existing red, green, gray, and alert surfaces.                                       |
| Backend | FastAPI skeleton with /login, /me, /nearby, /live-nearby, /search; API_CONTRACT.md describes /chat | Connect Android to these routes and add missing routes for contacts, profile, notifications, SOS, OTP, and Google auth. |

## Project Folder Map

| Path                                                                                    | Purpose                                                                                                                  |
|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| android/app/src/main/java/com/example/roadsos/MainActivity.kt                           | App entry point, splash/auth/permission/main flow routing.                                                               |
| android/app/src/main/java/com/example/roadsos/screens/navigation/MainContainerScreen.kt | Main screen router and bottom-tab state holder.                                                                          |
| android/app/src/main/java/com/example/roadsos/screens/home/HomeScreen.kt                | Dashboard, map mock, quick actions, SOS section, notifications, bottom navigation.                                       |
| android/app/src/main/java/com/example/roadsos/screens/services                          | Service list, filters, service detail view.                                                                              |
| android/app/src/main/java/com/example/roadsos/screens/contacts                          | Emergency contact list, add form, search, delete confirmation.                                                           |
| android/app/src/main/java/com/example/roadsos/screens/ai/AIAssistantScreen.kt           | Chat-style emergency assistant UI with simulated AI response/error.                                                      |
| android/app/src/main/java/com/example/roadsos/screens/profile/ProfileScreen.kt          | Editable profile fields, permissions shortcut, logout.                                                                   |
| android/app/src/main/java/com/example/roadsos/screens/permissions/PermissionScreen.kt   | Permission awareness/selection UI.                                                                                       |
| android/app/src/main/java/com/example/roadsos/ui/components/ErrorComponents.kt          | Reusable ErrorBanner, NoInternetBanner, LoadingView, EmptyStateCard, RetryButton.                                        |
| backend/app/api                                                                         | FastAPI route modules currently available for auth, nearby, search, health, download; chatbot route exists but is empty. |
| API_CONTRACT.md                                                                         | AI/backend contract for /chat classification, severity, facility ranking, and AI response.                               |

# App Flow

- Launch starts in MainActivity.kt and renders RoadSoSApp inside RoadSoSTheme.
- RoadSoSApp shows SplashScreen for 2.5 seconds using LaunchedEffect and delay.
- After splash, unauthenticated users see AuthScreen.
- Login success directly opens MainContainerScreen.
- Signup success opens PermissionScreen first; Save and Continue then opens MainContainerScreen.
- MainContainerScreen controls Home, Services, Service Detail, Contacts, Add Contact, AI, Profile, and Permissions.
- Android BackHandler returns most nested screens to their parent or to Home depending on the current route.

## Screen Specifications

### Splash Screen

| File           | screens/splash/SplashScreen.kt                                                                       |
|----------------|------------------------------------------------------------------------------------------------------|
| Current UI     | Branding, launch transition, animated SOS pulse, background image, dark overlay, progress indicator. |
| Backend work   | No backend dependency. Later can check persisted session/token while splash is visible.              |
| Error handling | If session refresh fails, route to AuthScreen and show auth error there.                             |

### Authentication

| File           | screens/auth/AuthScreen.kt                                                                                                                                  |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Signup/login toggle, name field for signup, phone field, OTP field, send OTP label, Google continuation button, local loading/error variables.              |
| Backend work   | Implement send OTP, verify OTP, login, signup, Google sign-in, token persistence, refresh token. Replace direct success callbacks with API result handling. |
| Error handling | Show Invalid OTP, network failure, rate limit, account exists/not found, Google failure, server unavailable. Use loading spinner during network calls.      |

### Permission Setup

| File           | screens/permissions/PermissionScreen.kt                                                                                                          |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Permission cards for location, phone calls, SMS, notifications; Save and Continue; back support.                                                 |
| Backend work   | Android should request actual runtime permissions and optionally send permission status to backend for user capability tracking.                 |
| Error handling | If permission is denied, show a non-blocking warning and allow degraded app mode. For permanently denied permissions, deep-link to app settings. |

### Home Dashboard

| File           | screens/home/HomeScreen.kt                                                                                                                                                                   |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Top header, GPS active indicator, notification popup, map mock, quick actions, SOS explainer, nearby services, no-internet banner, bottom nav.                                               |
| Backend work   | Fetch current user/location, nearby top services, notifications, live SOS status, map ETA, connectivity state. Quick actions should call service category endpoints or native phone intents. |
| Error handling | No internet banner; SOS failure banner; notification fetch failure should be quiet with retry; location missing should show permission/location CTA.                                         |

### Bottom Navigation + SOS

| File           | screens/home/HomeScreen.kt                                                                                                                                   |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Floating bottom nav with Home, Services, Contacts, AI and central SOS button. SOS confirmation dialog currently simulates failure.                           |
| Backend work   | Create SOS incident endpoint. On confirm send current location, user id, contacts, severity, selected service category, battery/network status if available. |
| Error handling | Handle SOS pending, sent, assigned, failed, canceled, resolved. Backend must return incident_id and status. UI should never silently fail an SOS.            |

## Services List

| File           | screens/services/ServicesScreen.kt                                                                                                                        |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Search, category filter chips, hardcoded service list, cards with distance/ETA/rating/call, empty state.                                                  |
| Backend work   | Connect to /nearby and /search. Support lat/lon/radius/type/query, distance sorting, availability, contact number, geolocation.                           |
| Error handling | Show LoadingView during fetch, EmptyStateCard for zero results, ErrorBanner + RetryButton for failed fetch. Cache last known results for offline display. |

## Service Detail

| File           | screens/services/ServiceDetailScreen.kt                                                                                                           |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Map-style header, back support, ETA chip, title/status, distance/ETA/rating, Directions/Call Now, location, facilities, emergency contact.        |
| Backend work   | Fetch service by id or pass full service object from list. Directions should open maps route. Call Now should dial backend-provided phone number. |
| Error handling | If service details fail, show cached basic details with retry. If phone unavailable, disable call and show explanation.                           |

## Contacts List

| File           | screens/contacts/ContactsScreen.kt                                                                                                                      |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Search by name/relation/number, add contact CTA, contact cards, priority chip, call button, delete confirmation.                                        |
| Backend work   | Add CRUD endpoints for contacts. Persist contacts per authenticated user. Contact call button should use ACTION_DIAL with stored number.                |
| Error handling | Create/update/delete failures need inline or banner errors. Duplicate numbers and invalid phone format should be validated server-side and client-side. |

## Add Contact

| File           | screens/contacts/AddContactScreen.kt                                                                                         |
|----------------|------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Name, phone, relation, priority dropdown, validation that blocks blank fields, back support.                                 |
| Backend work   | POST contact to backend, return server id. Priority values are Primary, Secondary, Medical, SOS.                             |
| Error handling | Show field-level validation for bad phone number, duplicate priority rules, backend failure, and loading state while saving. |

## AI Assistant

| File           | screens/ai/AIAssistantScreen.kt                                                                                                                                        |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Chat top bar, safety banner, LazyColumn messages, input bar with imePadding, dummy response, AI unavailable banner.                                                    |
| Backend work   | Connect to /chat contract. Send message, lat, lon, optional incident_id/context. Render ai_response, severity_detected, nearby facilities, and suspicious/test status. |
| Error handling | Show AI unavailable, timeout, rate limit, unsafe content, no location, empty response. Keep user message visible even when backend fails.                              |

## Profile

| File           | screens/profile/ProfileScreen.kt                                                                                              |
|----------------|-------------------------------------------------------------------------------------------------------------------------------|
| Current UI     | Profile icon, editable name/phone/email, permission settings link, save changes, logout.                                      |
| Backend work   | GET /me, PATCH /profile, logout/revoke token, persist profile locally. Current isEditing defaults true and Save locks fields. |
| Error handling | Validation for email/phone/name, save loading state, conflict errors, session expired leading to AuthScreen.                  |

# Reusable UI Components

| Component                                       | File                             | Purpose                                             | Backend use                                                          |
|-------------------------------------------------|----------------------------------|-----------------------------------------------------|----------------------------------------------------------------------|
| ErrorBanner(message)                            | ui/components/ErrorComponents.kt | Red warning card for failed operations.             | Use for API/network/domain errors that need user action.             |
| NoInternetBanner()                              | ui/components/ErrorComponents.kt | Full-width connectivity warning.                    | Bind to network monitor and offline repository state.                |
| LoadingView()                                   | ui/components/ErrorComponents.kt | Centered red Material spinner.                      | Use while API calls are in progress.                                 |
| EmptyStateCard(title, subtitle)                 | ui/components/ErrorComponents.kt | Reusable empty result card.                         | Use for no services, no contacts, no notifications, no chat history. |
| RetryButton(onRetry)                            | ui/components/ErrorComponents.kt | Consistent retry action.                            | Call repository fetch again after recoverable failures.              |
| BottomNavBar / SOSNavButton                     | screens/home/HomeScreen.kt       | Shared navigation and SOS confirmation entry point. | Connect center SOS to incident creation and status polling.          |
| AuthInputField / InputField / ProfileInputField | auth, contacts, profile screens  | Consistent styled text inputs.                      | Add validation states and field-level errors.                        |

## Recommended Error Handling Architecture

The backend should return predictable HTTP status codes and stable JSON error bodies. The Android side should translate them into a sealed UI error model rather than checking raw strings in each screen.

| Error category               | Backend response                 | Android UI behavior                                                                         |
|------------------------------|----------------------------------|---------------------------------------------------------------------------------------------|
| Network offline              | No server response               | NoInternetBanner, keep cached data, disable high-risk actions except native emergency call. |
| Timeout                      | Client timeout or 504            | ErrorBanner with RetryButton; keep user-entered form/chat text.                             |
| Unauthorized/session expired | 401                              | Clear token, show AuthScreen, preserve non-sensitive local state when possible.             |
| Permission missing           | 403 or app-side permission state | Show permission CTA and explain degraded mode.                                              |
| Validation                   | 400/422 with field map           | Show field-level errors near input and block submit.                                        |
| Not found                    | 404                              | Service/contact unavailable state with back navigation.                                     |
| Rate limited                 | 429                              | Show cooldown message; disable button temporarily.                                          |
| Server failure               | 500/503                          | ErrorBanner with support-safe wording; log request id if provided.                          |
| SOS dispatch failed          | 409/500/503 with incident status | Prominent banner/dialog; offer retry and direct emergency calling.                          |
| AI unavailable               | 503 or AI timeout                | Keep chat message, show AI unavailable banner, optionally offer canned first-aid guidance.  |

## Suggested API Contracts

| Endpoint              | Request                    | Frontend usage                                                              |
|-----------------------|----------------------------|-----------------------------------------------------------------------------|
| POST /auth/send-otp   | { phone }                  | Send OTP; return request_id, expires_at, retry_after.                       |
| POST /auth/verify-otp | { phone, otp, request_id } | Verify login/signup; return user, access_token, refresh_token, is_new_user. |
| POST /auth/google     | { id_token }               | Google auth; return same token payload.                                     |
| GET /me               | Bearer token               | Return current profile; backend already has a basic route.                  |
| PATCH /profile        | { name, phone, email }     | Update editable profile fields.                                             |
| GET /nearby           | lat, lon, radius, type     | Return nearby services; backend route exists.                               |
| GET /search           | q                          | Return services matching query; backend route exists.                       |

| Endpoint               | Request                                     | Frontend usage                                                                        |
|------------------------|---------------------------------------------|---------------------------------------------------------------------------------------|
| GET /services/{id}     | service id                                  | Return detail page data: address, facilities, phone, open status, ETA metadata.       |
| GET /contacts          | Bearer token                                | Return user's emergency contacts.                                                     |
| POST /contacts         | { name, relation, phone, priority }         | Create contact and return id.                                                         |
| DELETE /contacts/{id}  | contact id                                  | Delete after confirmation.                                                            |
| POST /sos              | { lat, lon, message?, contacts, severity? } | Create SOS incident; return incident_id, status, notified contacts, assigned service. |
| GET /sos/{incident_id} | incident id                                 | Poll incident state: pending, dispatched, assigned, resolved, failed.                 |
| GET /notifications     | Bearer token                                | Return emergency alerts and status updates.                                           |
| POST /chat             | { message, lat, lon, incident_id? }         | Use API_CONTRACT.md: return facilities, ai_response, severity_detected.               |

# Data Models Android Will Need

| Model            | Important fields                                                                                                             |
|------------------|------------------------------------------------------------------------------------------------------------------------------|
| User             | id, name, phone, email, avatar_url, created_at, permission_status.                                                           |
| EmergencyService | id, name, type, distance_km, eta_minutes, rating, phone, address, lat, lon, open_now, facilities, availability_status.       |
| EmergencyContact | id, name, relation, phone, priority, verified, created_at.                                                                   |
| Notification     | id, title, body, type, severity, created_at, read, incident_id.                                                              |
| SOSIncident      | id, user_id, lat, lon, status, assigned_service_id, eta_minutes, contacts_notified, created_at, resolved_at, failure_reason. |
| ChatMessage      | id, role(user/assistant/system), text, time, severity_detected, related_facilities, error_state.                             |
| ApiError         | code, message, field_errors, request_id, retry_after_seconds.                                                                |

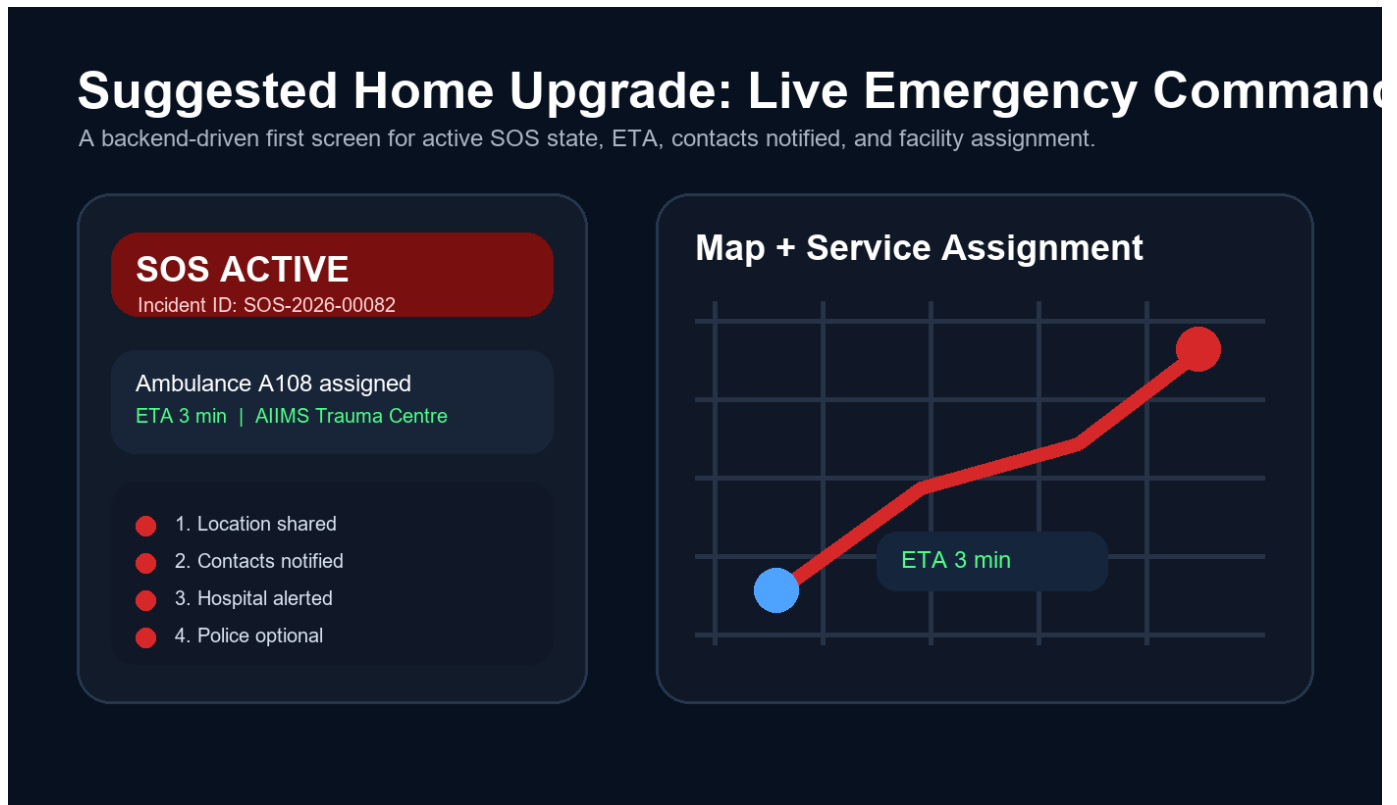
## Backend Integration Steps

- Add Android networking dependencies such as Retrofit or Ktor, kotlinox.serialization/Moshi/Gson, and OkHttp logging for debug builds.
- Create DTOs matching backend responses and domain models matching UI needs.
- Introduce repositories: AuthRepository, ServiceRepository, ContactRepository, SOSRepository, ChatRepository, ProfileRepository, NotificationRepository.
- Move per-screen state from remember into ViewModels with sealed states: Loading, Success, Empty, Error.
- Persist access/refresh tokens in EncryptedSharedPreferences or DataStore.
- Add network connectivity monitoring and bind it to NoInternetBanner.
- Replace hardcoded lists in ServicesScreen, ContactsScreen, HomeScreen, NotificationPopup, and AIAssistantScreen.
- Add runtime permission requests using ActivityResultContracts for location/call/SMS/notification.
- Wire native intents: ACTION\_DIAL for call buttons and maps intent for Directions.
- Add SOS status polling or realtime channel, because emergency dispatch should update without manual refresh.

## Current Gaps and Improvement Scope

| Area          | Current state                        | Recommended improvement                                                              |
|---------------|--------------------------------------|--------------------------------------------------------------------------------------|
| Navigation    | Manual enum router                   | Use Navigation Compose for back stack, args, and deep links.                         |
| State         | remember state inside Composables    | Move business state to ViewModels and repositories.                                  |
| Auth          | Direct callback success; no real OTP | Implement real OTP/Google auth, token storage, session refresh.                      |
| Location      | Static text and mock map             | Use fused location provider and maps SDK or backend map tiles/ETA.                   |
| Services      | Hardcoded local list                 | Fetch dynamic nearby/live services with cache and retry.                             |
| Contacts      | In-memory list                       | Persist per user; sync with backend; support edit/reorder/verification.              |
| SOS           | Confirmation then simulated failure  | Create incident lifecycle with retry, fallback call, notifications, and audit log.   |
| AI            | Dummy response and error             | Connect to /chat, severity-aware UI, preserve history.                               |
| Notifications | Static popup data                    | Fetch/push notifications; mark read; link to incident/service detail.                |
| Accessibility | contentDescription mostly null       | Add meaningful descriptions for emergency controls.                                  |
| Security      | No token/data storage yet            | Encrypt tokens, avoid logging sensitive emergency data.                              |
| Testing       | Default sample tests only            | Add ViewModel unit tests, API mapper tests, and Compose UI tests for critical flows. |

## Suggested UI Enhancements With Mockup



Recommended additions: an active SOS command panel, incident timeline, assigned ambulance/hospital details, contact notification status, and backend-generated incident ID. This helps the user understand that the emergency request is actually moving through the system, and gives the backend a clear set of states to expose.

- Add an Active Incident banner on Home when `/sos/{id}` is pending, assigned, or failed.
- Add a severity chip from `/chat severity_detected` or SOS triage: serious, minor, default.
- Add a compact notification inbox count and read/unread state from backend.
- Add service availability badges: Open, Busy, No beds, Offline, Verified.
- Add contact delivery states: SMS sent, call attempted, failed, acknowledged.
- Add safe fallback actions: Call 108, Call 100, Share location via SMS/WhatsApp when backend is unreachable.

# Screen-to-Backend Ownership Matrix

| Frontend screen/component | Backend/API owner should provide                      | Acceptance criteria                                                                |
|---------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------|
| AuthScreen                | OTP send/verify, Google auth, token refresh           | Invalid OTP, loading, success, and session expiry paths all testable.              |
| PermissionScreen          | Optional permission-status save endpoint              | Backend knows whether location/SMS/call capabilities are available.                |
| HomeScreen TopSection     | User profile, GPS/location status, notification count | Header shows real user/location state.                                             |
| NotificationPopup         | GET /notifications and optional mark-read             | Alerts are dynamic, sorted newest first, and connected to incident/service detail. |
| LocationMapCard           | ETA, current location, assigned destination           | Map card can show real coordinates and current route summary.                      |
| QuickActionsSection       | Category service lookup and emergency phone numbers   | Ambulance/Police/Hospital/Towing cards open relevant action/data.                  |
| BottomNavBar SOS          | POST /sos and GET /sos/{id}                           | SOS creates incident, returns id, and updates through lifecycle.                   |
| ServicesScreen            | GET /nearby, GET /search                              | Filter/search results come from backend and handle empty/error/loading states.     |
| ServiceDetailScreen       | GET /services/{id}                                    | Directions/call/location/facilities use real data.                                 |
| ContactsScreen            | Contacts CRUD                                         | Contacts persist across app restarts and devices.                                  |
| AIAssistantScreen         | POST /chat                                            | AI reply, severity, and ranked facilities render from backend response.            |
| ProfileScreen             | GET /me, PATCH /profile, logout                       | Profile is editable, persisted, and validated.                                     |

## Implementation Notes for Backend Developer

- Keep response shapes stable and include a top-level success boolean only if it is used consistently. Prefer proper HTTP codes plus a structured error body.
- Every protected route should require the same Bearer token verification used by /me and /nearby.
- Include request\_id in error responses so Android can show/log support-safe diagnostics without exposing internals.
- For /nearby, return numeric distance\_km and eta\_minutes separately; Android can format them as '0.8 km' and '4 min'.
- For /chat, follow API\_CONTRACT.md and include severity\_detected so Android can highlight serious emergencies.
- For SOS, design idempotency. If the user taps Send SOS twice during a bad connection, the backend should not create duplicate incidents.
- For contacts and profile, validate phone numbers in E.164 where possible, but allow local emergency numbers such as 100 and 108.
- Avoid sending sensitive personal data in AI prompts unless required; send only emergency context, location, and nearby services needed for guidance.

## Final Handoff Summary

The frontend has completed the core emergency UX: splash/auth flow, signup permissions, dashboard, simulated map, quick actions, SOS confirmation, bottom navigation, services list and detail, contact management, AI chat, profile editing, reusable error/empty/loading components, and dark emergency-first visual system. The next major milestone is backend binding: replace local state with API-backed ViewModels, introduce persistence and permissions, then harden SOS as the highest-priority workflow.